Ajeitando normalização pelo conceitual

Etapa 5: Mapeamento de tipos de relacionamento binário 1:N

- Assume (Funcionário - Cargo) 1:N

  Um funcionário deve ter um cargo; cada cargo pode pertencer a vários funcionários - atualiza Funcionário

  Funcionario (cpf_usuario*, matricula, cpf_supervisor*, cod_cargo*, data_admissao, data_saida)

  cpf_usuario referencia Usuario (cpf)

  cpf_supervisor referencia Funcionario (cpf)

  cod_cargo referencia Cargo (cod_cargo)

  ↓

Etapa 6: Mapeamento de tipos de relacionamento binário M:N

- Assume (Funcionário - Cargo) N:N

  Todo funcionário deve assumir obrigatoriamente um cargo em sua admissão. Ao longo de sua trajetória na empresa, um funcionário pode assumir múltiplos cargos, porém ocupando estritamente um cargo por vez; cada cargo pode pertencer a vários funcionários - atualiza Funcionário - nova tabela

  Assume (cpf_funcionário*, cargo*, data_admissao, data_saida)

  cpf_funcionario referencia Funcionário(cpf_usuario)

  cargo referencia Cargo(cod_cargo)

  Problema percebido:

  Se um funcionário assumir um cargo gerente, por exemplo, dps virar diretor, e dps tentar voltar a ser gerente, isso não vai ser possível, pois já existe exatamente a chave-primária que representa esse caso anterior - data_admissao deve virar parte da chave primária tb, esse exemplo fala disso

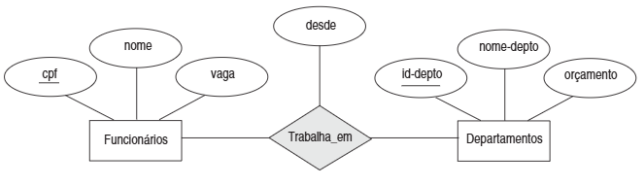


Figura 2.2 O conjunto de relacionamentos Trabalha_em.

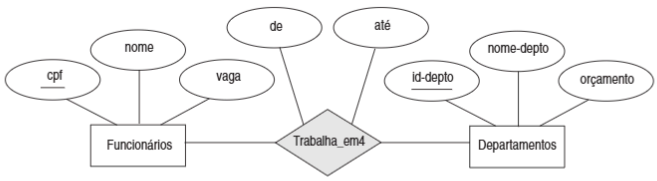


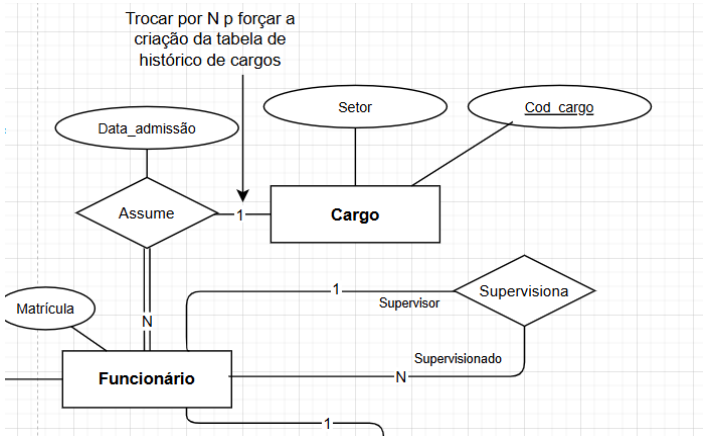
Figura 2.14 O conjunto de relacionamentos Trabalha_em4.

O Trabalha_em4 difere do conjunto de relacionamentos Trabalha_em da Figura 2.2 apenas pelo fato de ter os atributos *de* e *até* em vez de *desde*. Intuitivamente, ele registra o intervalo durante o qual um funcionário trabalha para um departamento. Agora suponha que seja possível que um empregado trabalhe em determinado departamento por mais do que um período.

Essa possibilidade não é permitida pela semântica do diagrama ER, porque um relacionamento é identificado univocamente pelas entidades participantes (lembre-se

Colocar as datas de intervalo ao invés só da data de admissão facilitaria a busca em questão de consultas mesmo lá na frente, se quisesse saber qnd o funcionário saiu da empresa, n ia ter como saber realmente, e qnd fosse fosse para saber a saída de um cargo p outro, ia ter que puxar o próximo cargo e daí deduzir a saída do anterior.

Vou fazer alterações temporárias no eer pensando nisso, e colocar um atributo de saída em assume



Relações finais normalizadas

- Usuario (cpf, nome, cep, n_residência, complemento, tipo_endereco))
- Telefone_Usuario (cpf_usuario*, numero)
  cpf_usuario referencia Usuario(cpf)
- Cliente (cpf_usuario*, tipo_assinatura)
  cpf_usuario refencia Usuário(cpf)
- Funcionario (cpf_usuario*, matricula, cpf_supervisor*)
  cpf_supervisor referencia Funcionario (cpf_usuario)
  cpf_usuario referencia Usuario(cpf)
- Reclamacao (cod_reclamacao, cod_pedido*, cpf_funcionario*, status, classificacao, descricao, data_resolucao)
  cod_pedido referencia Pedido (cod_pedido)
  cpf_funcionario referencia Funcionario (cpf_usuario)
- Categoria (cod_categoria, nome, descricao)

- Promocao (cod_promocao, percentual, periodo_ativo)
- Promocao (cod_promocao, percentual, data_inicio, data_fim)
- Entrega (cod_rastreio, cnpj_transportadora*, data_postagem, data_estimada, data_hora_entrega)
  cnpj_transportadora referencia Transportadora (cnpj)
- Transportadora (cnpj, nome, serviço)
- Telefone_Transportadora (cnpj_transportadora*, numero)
  cnpj_transportadora referencia Transportadora (cnpj)
- Produto (cod_produto, nome, preco, quantidade_estoque)

- Pagamento (cod_transacao, data, meio_pagamento,status)
- Pedido (cod_pedido, cpf_cliente*, cod_transacao_pagamento*, data_hora, status)
  cod_transacao_pagamento referencia Pagamento (cod_transacao)
  cpf_cliente referencia Cliente (cpf_usuario)

  sobre Pedido, ele está em relacionamento 1:1 com obrigatoriedade em ambos os lados com Pagamento, e eu vi no livro isso:

2. **Técnica de relação mesclada:** um mapeamento alternativo de um tipo de relacionamento 1:1 é mesclar os dois tipos de entidade e o relacionamento em uma única relação. Isso é possível quando *ambas as participações são totais*, pois indicaria que as duas tabelas terão exatamente o mesmo número de tuplas o tempo inteiro.

e o gemini exemplificou aqui:

- **A Mecânica do Erro:**
  1. Como a participação do Pedido é total, a sua Chave Estrangeira `cod_transacao_pagamento` precisa ser definida como `NOT NULL` no comando `CREATE TABLE`. +1
  2. Um cliente clica em "Comprar". O sistema tenta fazer um `INSERT` na tabela `Pedido`
  3. Para o `INSERT` em `Pedido` funcionar, o SGBD exige que o `cod_transacao_pagamento` informado já exista na tabela `Pagamento` (Restrição de Integridade Referencial). +1
  4. O sistema tenta, então, fazer o `INSERT` na tabela `Pagamento` primeiro.
  5. Porém, no mundo real, a transação de pagamento (cartão de crédito) precisa dos dados do `Pedido` (valor total, itens, etc.) para ser processada pela operadora de cartão.

O Resultado: Você tem duas tabelas. Uma exige que o dado da outra já exista. Se a chave for `NOT NULL`, você não consegue inserir nenhuma das duas via comandos DML simples. A solução exigiria artifícios avançados de SQL (como `DEFERRED CONSTRAINTS`, que atrasam a verificação da FK para o final da transação), o que não é ideal se pudermos resolver isso no

Para solucionar isso, a ideia é justamente colocar tudo numa mesma tabela:

Pedido_Pagamento(cod_pedido, cpf_cliente*, data_hora, status_pedido, cod_transacao, data, meio_pagamento, status_pagamento)
cpf_cliente referencia Cliente (cpf_usuario)

cod_transacao deixaria de ser chave primária, já que cod_pedido já identificaria tudo sozinho.
mas sinceramente n gosto mt de fazer isso, e acho que fica inclusive mais lógico se a gnt só mudar lá no diagrama/modelo conceitual que apenas o pagamento é obrigatório, e n o pedido, fica inclusive mais realista e tb mais fácil de modelar

Nesse caso, como pagamento depende de pedido p existir, faz mais sentido ele armazenar a chave estrangeira em pagamento:

- Pagamento (cod_transacao, data, meio_pagamento,status, pedido*)
  pedido referencia Pedido(cod_pedido)
- Pedido (cod_pedido, cpf_cliente*, data_hora, status)
  cpf_cliente referencia Cliente (cpf_usuario)

- Atribui (cpf_cliente*, cod_promocao*, cod_categoria*)
  cpf_cliente referencia Cliente (cpf_usuario)
  cod_promocao referencia Promocao (cod_promocao)
  cod_categoria referencia Categoria (cod_categoria)
- Item do Pedido (cod_pedido*, cod_produto*, cod_rastreio_entrega*, cod_promocao*, quantidade, valor_unitario, nota_avaliacao, comentario)
  cod_pedido referencia Pedido (cod_pedido)
  cod_produto referencia Produto (cod_produto)
  cod_rastreio_entrega referencia Entrega (cod_rastreio)
  cpf_cliente_avaliador referencia Cliente (cpf_usuario)
  cod_promocao referencia Promocao (cod_promocao)
- Pertence (cod_categoria*, cod_produto*)
  cod_categoria referencia Categoria (cod_categoria)
  cod_produto referencia Produto (cod_produto)
- Histórico de Cargos~~Assume~~ (cpf_funcionário*, cargo*, data_admissã~~-~~ao, data_saida)
  cpf_funcionario referencia Funcionário(cpf_usuario)
  cargo referencia Cargo(cod_cargo)

- Cargo (cod_cargo, setor, nome)
- Historico de Precos (cod_produto_preco*, data_vigencia, valor)
  cod_produto_preco referencia Produto(cod_produto)